National University of Lesotho

Department of Mathematics and Computer Science

CS4433 Software Engineering

Individual Final Project

Moderated Learning Management System

Project Report

Student Name: MAKHOTSO RAMOCHELA

Student Number: 202321185

Git repository: https://github.com/makhotsoramochela/Software-Engineering-Project.git

Version of the system: 1.1

Abstract

This report describes the design, building, testing and release of a Moderated Learning Management System for a secondary school with about 800 students. Students submit work, lecturers mark it, and the Head of Department checks every mark and releases it. A student sees a mark only after it is released. Late work is accepted, but loses 10% of the maximum mark for each day late, up to three days, and then receives zero. The system is a console program written in C++17. It keeps all its data in one SQLite file, stores passwords as Argon2id hashes, and works offline on one old school computer.

The project followed the Waterfall model with V-Model testing over six weeks. It has 40 requirements, a four-layer design with six design patterns, and a database of 12 tables. Version 1.0 passed 36 automated unit and integration tests, 95 system tests and 7 acceptance tests, with 85.7% line coverage of the business layer. After release the school asked for an extension for a whole class (change request CR1), and this was released as version 1.1 with all earlier tests still passing. The report also lists what is not yet proved and the professional, legal and ethical issues of the system.

Table of contents

2. Introduction 3
3. Project management plan 3
4. Requirements engineering 3
5. Software design 4
6. Implementation 4
7. Verification and validation 5
8. Configuration management and quality assurance 5
9. Deployment 6
10. Maintenance (change request CR1) 6
11. Professional, legal and ethical issues 7
12. Conclusion 7
References 8

2. Introduction

A secondary school with about 800 students and about 100 staff needs a system to manage courses, assignments and marking. It has four kinds of user: students, lecturers, Heads of Department (HoDs) and an administrator, and each may do only what the role allows. The school's computers are old and its internet is unreliable, so the system must work offline. The central idea is moderated marking: a lecturer marks the work, the HoD checks the mark and may change or return it, and only then is it released to the student.

The aim of the project was to deliver this system as a real software project, from planning to maintenance, with documents, code and tests that describe the same system. The work followed the assignment brief (National University of Lesotho, 2026) and the Waterfall model with V-Model testing (Sommerville, 2016). Each phase ended with a deliverable, and three gates checked the work. Sections 3 to 10 follow the phases. Section 11 discusses professional, legal and ethical issues.

3. Project management plan

Schedule and effort. I worked alone for six weeks, from 28 September to 6 November 2026. That is 30 working days and 180 hours at 6 hours a day. The plan uses 168 hours and leaves 12 spare hours. I split the project into 7 phases and 28 tasks, and each task has one deliverable. Because there is one developer, the tasks run one after another, so the only protection against delay is the spare time before each gate: Gate 1 (requirements, 9 October), Gate 2 (design, 16 October) and Gate 3 with release v1.0 (3 November). Release v1.1 was planned for 5 November. The critical path is the last stage, tasks 7.1 to 7.5 (the change request and the final report), which has no spare time [D1].

Feasibility. The five areas were all judged feasible. Technically, the tools are free and well known: C++17, CMake, SQLite, libsodium and GoogleTest. SQLite is one file with no server, which suits old computers. Economically there is no licence or hardware cost. Operationally, menus that show only the actions of the user's role keep the system simple. Legally, the Lesotho Data Protection Act, 2011 applies because the system holds learners' data (Lesotho, 2011). The schedule is the weak point: six weeks instead of the seven in the brief, and only 12 spare hours. I recommended going ahead on four conditions: the Must requirements are fixed at Gate 1; the repository, build and CI are set up in Week 1; extensions are designed with CR1 in mind; and the risk register is reviewed at every gate.

Risks. The register has ten risks (four technical, three project and three people risks). Each is scored as likelihood times impact on a scale of 1 to 3 (Boehm, 1991). The highest is R1, taking longer than planned because this is my first large C++ project (score 9). Seven risks score 6 or more. The main actions are small build steps, weekly checks of hours, tests written with the code, a traceability matrix against mismatched artefacts, and a MoSCoW scope list against scope growth. The register is reviewed at each gate, and the updated register is in the appendices.

4. Requirements engineering

Stakeholders. Six stakeholders were identified: the Principal, the HoD, lecturers, students, the IT officer and the system administrator. The Principal and the HoD have high influence and high interest, so their statements win when there is a conflict. Fourteen statements (C1 to C14) were found to be conflicting, unclear or incomplete, and each was settled with a written decision. For example, the lecturer wants marks released at once but the Principal wants every mark checked first, so marks are hidden until the HoD releases them. The HoD wants zero after three days but students do not want late work rejected, so late work is always accepted and saved, the student sees the penalty before saving, and the mark is zero only after 72 hours. An extension gives a student a later deadline and the penalty is counted from it (from version 1.1 a lecturer can give it to a whole class).

Requirements. The SRS follows ISO/IEC/IEEE 29148 (2018) and has 40 requirements: 26 functional and 14 non-functional. By MoSCoW priority, 32 are Must, 7 are Should and 1 is Could. Every requirement has an identifier, a source and a way to check it. The non-functional requirements are measurable. For example, logging in must take at most 2 seconds in at least 29 of 30 attempts with 800

student accounts stored (NFR-01), and one password check must take between 100 and 500 milliseconds (NFR-03). Ten business rules (BR-01 to BR-10) fix the details: the late penalty, hidden marks until release, resubmission only before the deadline, extensions, the audit log, the allowed status changes, moderation by department, the password policy, the mark format and who may see which data.

Use cases. The use case model has 20 use cases for the four actors, with include and extend relationships. For example, marking includes the penalty calculation and the audit entry, and resubmitting extends submitting [D2]. Every functional requirement is linked to a use case. Three of them (including Submit assignment) have full specifications with main, alternative and exception flows.

5. Software design

Architecture. The system uses a layered architecture with four layers: presentation (console menus), business logic (services, rules and domain classes), data access (SQLite repositories, the clock and the password hasher) and the platform (SQLite, libsodium, the operating system) [D3]. A layer uses only the layer below. The business layer owns the interfaces that the data layer implements, so it never depends on SQLite or libsodium (the Dependency Inversion Principle; Martin, 2003). Ten architecture rules (AR-1 to AR-10) turn this into checks. Examples are: menus call only services; all SQL is in the repositories and is parameterised; every service method checks the role first; each change is one transaction; time is read only through a Clock; and the audit log can only be added to. I considered the Model-View-Controller style, but it has no separate home for the rules, which would end up in the controllers and be hard to test without the console.

Classes and patterns. The design has an abstract User class with four subclasses, eight services, six repository interfaces and the domain classes [D4]. Six patterns are used, each for a reason. Strategy holds the late-penalty rule, the most argued rule, so a new rule is a new class. Repository hides SQLite. A simple Factory creates the right user subclass. An Adapter puts libsodium behind a PasswordHasher interface. Template Method gives all menus the same loop and error handling. Unit of Work with RAII makes a mark, its status change and its audit entry succeed or fail together (Gamma et al., 1994; Fowler, 2002).

Behaviour. A submission has six statuses: Submitted, Late, Marked, Returned, Moderated and Released. One operation changes status and accepts only the seven allowed changes, so any other change is refused [D5]. A student sees only Submitted or Late until the mark is Released. Activity and sequence diagrams show the flow from submission to released mark, the submit path with the late and resubmission cases, moderation and release [D6] [D7].

Database. The database has 12 tables in third normal form [D8]. The four user classes share one table, because they have almost the same columns and one table makes login and role checks simple. Each submission keeps all its versions. Dates are real dates checked by constraints, and triggers stop the audit log from being changed. A few values, such as the days late, are saved on purpose and are written in the same transaction as the fact they come from. The class-to-table mapping and the data dictionary are in the design document.

6. Implementation

The system is about 4,100 lines of C++17 in three folders that match the presentation, business and data layers, built with CMake. The build enforces the layer rules: the business library has no include path to SQLite or libsodium, and the menu library links only to the business library, so a menu cannot run SQL. The code compiles with no warnings under strict compiler settings and follows a short written coding standard.

Key decisions. (1) Every change that touches more than one table is one transaction that rolls back by itself if an error is thrown. (2) The clock is injected, so tests can set any time and check the late-penalty boundaries, such as exactly 24 or 72 hours. (3) Late days are rounded up: 1 to 3 days late loses n times 10% of the maximum mark, and more than 3 days gives a final mark of zero. (4) A student's visible status follows a rule in the domain class, not in the menus, so a mark cannot leak by mistake. (5) The

database schema is built into the program, so installation needs one file less, and a database file with another version number, or one that belongs to another program, is refused and not changed.

Security. Passwords are stored only as Argon2id hashes with a random salt, made by libsodium with 4 passes and 64 MiB of memory. One check takes about 120 ms, inside the 100 to 500 ms target. After five wrong passwords the account is locked for five minutes, and the error message does not say whether the username or the password was wrong. Every service method checks the role and the user's own course, department or work before it does anything. All SQL values are bound as parameters. Every change of a final mark writes one audit entry with the old mark, the new mark, the person, the time and a reason, and the program has no way to edit or delete it. Persistence is one SQLite file that also holds the submitted files (at most 5 MB each) and all their versions.

Differences from the design. The code follows the design, with 18 small differences that are listed with reasons in the repository (for example extra helper classes, and tests that use the real SQLite repositories on an in-memory database instead of fake ones). All 18 differences, and the change for CR1, are written into the design document (version 4, Section 7).

7. Verification and validation

Testing followed the V-Model, with each level matched to a design level. Table 1 shows the results for version 1.0.

Table 1: Test results for version 1.0

Level	Tests	Result
Unit (rules)	9 GoogleTest tests	All pass
Integration (workflow and security)	27 GoogleTest tests on real SQLite, fake clock	All pass (36 in total)
System	95 tests from the use cases, with exception flows	95 of 95 pass
Acceptance	7 tests of the case study statements	7 of 7 pass; 2 more need people
Scripted session and wrong input	1 full session (14 checks); 60 sessions of random input	No failures, no crashes
Coverage of the business layer	gcov, target 70%	85.7% (824 of 961 lines)

The timing tests on the test computer show login at 119.7 ms on average (worst 140.6 ms) against a limit of 2 seconds, saving a 5 MB file in 20.8 ms, and listing 800 submissions in 2.4 ms. Six defects were found and fixed, and none is open. The most important were that the password check took 94 ms and was too fast to meet NFR-03, and that the program accepted and changed a database file that belonged to another program. When the final tests were run on the code from before the fixes, three failed, which shows that the defects were real. Some failures were mistakes in the test scripts, and they were corrected and recorded.

8. Configuration management and quality assurance

Configuration management. The code is in Git. The branch `main` always builds and passes the tests. Each piece of work is done on a feature branch and merged with a merge commit (`git merge --no-ff`), so the history shows where each piece began and ended. Each commit does one job. Annotated tags mark each release: v0.4.0, v0.5.0 and v0.6.0 for phases 4 to 6, v1.0 for the first release and v1.1 for CR1. The merges so far were done locally, and on GitHub they would be pull requests. A GitHub Actions pipeline runs on every push and pull request. Its eight steps are install, build, unit and integration tests, system tests, the scripted and wrong-input tests, coverage (it fails below 70%) and static analysis. All steps pass when run on my computer with a script that runs the same commands. The pipeline has not yet run on GitHub, and the first real run will be added as evidence.

Quality assurance. clang-tidy found no problems, and clang-format reports none. cppcheck gave 24 style and performance hints. I fixed 8 and kept 16 with a written reason each, for example the loops that

are clearer than library calls, and four operations that belong to the design. One suggestion would have introduced a bug, because a reference changed the old mark that the audit entry needs, and a test caught it. The metrics show an average cyclomatic complexity of 2.2 per function. I set a limit of 10, and seven functions were above it: five were split into smaller functions, and two menu functions that are plain switch statements were kept. A self code review used a 14-item checklist covering the layers, SQL, role checks, passwords, memory, casts, magic numbers, complexity and duplicate code. It led to a named constant for a number typed 19 times, the splitting of long functions and the removal of a repeated block.

9. Deployment

Version 1.0 is installed on one school computer running Ubuntu Linux 24.04 [D9]. It has three parts: the program `lms`, the two libraries SQLite and libsodium, and the data file `lms.db`. A backup is a copy of that one file, made when the program is stopped. The internet is needed only once, to install the build tools. The installation guide for a school technician has nine steps: install the tools, get the code, build, copy the program, make a data folder, start the program, create the first Administrator (there is no default password), check the login and make a second Administrator. A build on a clean copy took about 31 seconds. The package also has a user manual for each of the four roles, a table of common problems and the release notes. The release was tagged v1.0. The timed trial with a technician and the trial of the manuals with five people have not been done, so NFR-07 and NFR-11 are open.

10. Maintenance (change request CR1)

After release the school asked: "A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list."

Classification. This is perfective maintenance: the program works correctly, and the school wants new functionality. It is not corrective because nothing is wrong, not adaptive because the environment is the same, and not preventive because it is not done to avoid a future fault. The SRS had listed "an extension for a whole class" as Won't have in version 1.0, so the change was also recorded in the requirements change log.

Impact analysis. I compared three solutions. A) Save one extension for each enrolled student in one transaction. B) Add a class-level extension row with an empty student, which needs a schema change and a change to the deadline rule. C) Let the menu call the single-student operation in a loop, which breaks the rule that one change is one transaction. I chose A because it needs no schema change and reuses the existing deadline rule and BR-04. The affected items are: requirements FR-10, FR-15 and BR-04; use case UC-07; the AssignmentService class and its Figure 3.5, a new sequence diagram (Figure 4.8), the StudentAssignmentRow type and a new ClassExtensionResult type; the lecturer and student menus; the traceability matrix, the manuals and the release notes. No table changes. The risks were breaking the single-student extension and the late-penalty recalculation, so the existing tests for them were chosen as the first regression checks.

Implementation (v1.1). The Lecturer menu has a new last choice, 8, "Give an extension to the whole class", so the numbers of the old choices did not change. The lecturer types the assignment, the new deadline and a reason, and confirms. Each enrolled student gets the extension by the same rules as before. A student who already has the same or a later deadline, or whose work is marked, is skipped, and the lecturer is told how many were skipped. Late work is recalculated. If nobody can get the extension, nothing is saved. The student's list has a new column, "Original deadline", that shows the first deadline when it was extended. The change touched 8 source files (108 lines added, 15 removed). The SRS and the design document were updated to new versions.

Regression testing. Table 2 compares the two versions. The old tests were not changed, and all of them pass on version 1.1. The 8 new system tests fail on version 1.0 and pass on version 1.1, which shows that they test the change. Coverage rose to 86.2%, and static analysis is still clean. One limit is that a student who is enrolled after the extension does not get it, and the lecturer must give that student an individual extension.

Table 2: Version 1.0 compared with version 1.1

Check	Version 1.0	Version 1.1
Unit and integration tests	36 pass	44 pass (36 old, 8 new)
System tests	95 pass	103 pass (95 old, 8 new)
Scripted session, wrong-input sessions	14 checks, 60 sessions	14 checks, 60 sessions, all pass
Coverage of the business layer	85.7%	86.2% (856 of 993 lines)
Static analysis	No findings	No findings

11. Professional, legal and ethical issues

Data protection. The system holds the names, enrolments, submissions and marks of learners, and many are under 18, so the Lesotho Data Protection Act, 2011 applies (Lesotho, 2011). The design collects only what is needed: a user has a username, a full name, a role and a password hash, with no contact details or identity numbers. Each role sees only its own data. Users and courses are deactivated and not deleted, so records stay for audit. The school must decide how long to keep them and how to answer a request to see or correct data, and it must check what the Act requires of it. I am not a lawyer, and this report is not legal advice.

Security of assessment records. The design protects marks with role checks in every service, hidden marks until release, Argon2id hashes, parameterized SQL, an audit log that the program cannot change and a lock after five wrong passwords. The weaknesses are real. The data file is not encrypted, and a person with access to the computer can copy it or edit it with another tool. The school should limit access to the computer and the file, use disk encryption and keep backups safe. The program has no way for users to change their own password, and a lost password of the only Administrator cannot be recovered, so two Administrators are advised.

Accessibility. The program uses text only, never color alone, and every function works with the keyboard (NFR-12). Menus are short numbered lists, and the terminal's own font and size settings can be used. However, I did not test it with screen readers or with users who have disabilities, and it is in English only, so accessibility is only partly proved.

Fairness of the late penalty. The rule comes from the HoD: 10% of the maximum mark for each day late for up to three days, then zero. It is fair in some ways: the same rule applies to everyone, the student sees the penalty before saving, late work is still accepted, marked and given feedback, extensions need a reason and are recorded with who gave them, and the HoD can adjust a mark only with a recorded reason. It is harsh in others: part of a day counts as a whole day, and the fall to zero after 72 hours is a sharp cliff. These are policy choices of the school and I implemented them, but I record them so that the school can review them. Class extensions make it easier to treat a whole class equally after a shared problem such as a power cut.

12. Conclusion

The project delivered a working moderated LMS in two releases. It meets the case study: roles with limited rights, moderated release of marks, a fixed late-penalty rule, resubmission, written feedback, offline use and protected passwords. The artefacts agree with each other, and every functional requirement is traced to design, code and tests. The layered design, the use of interfaces and the injected clock made the system easy to test, and testing found real defects early. The change request showed that the design could take a new feature in a small, local change.

The main gaps are these. The system was tested on Ubuntu only, and not on Windows or on the slow reference computer. The usability trial (NFR-07), the timed installation (NFR-11) and the first GitHub Actions run are still to be done. Future work could add a password change by the user, encryption of the data file, class extensions for students who enroll later, and tests with users who have disabilities.

References

Boehm, B.W. (1991) 'Software risk management: principles and practices', IEEE Software, 8(1), pp. 32-41.

Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley.

ISO/IEC/IEEE (2018) ISO/IEC/IEEE 29148:2018 Systems and software engineering: Life cycle processes: Requirements engineering. Geneva: ISO.

Lesotho (2011) Data Protection Act, 2011. Maseru: Government of Lesotho.

Martin, R.C. (2003) Agile Software Development: Principles, Patterns, and Practices. Upper Saddle River: Prentice Hall.

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson.